

Assignment 05: Comparing the C++ and OCaml Key-Value Stores

1. Which operations in your C++ program have side effects?

The C++ version has several side effects because it changes the same Store object while the program runs. The set, remove, and load functions change the map inside the store. Save writes to a file and load reads from one. The program also uses `std::cin` and `std::cout` for input and output. The transaction destructor can also restore the previous store if it is not committed.

2. Which operations in your OCaml program have side effects?

The OCaml version has fewer side effects because most of the store functions do not change existing data. The main side effects come from input and output. The save function writes to a file, while load and its helper functions read from a file. The command loop also reads from standard input and prints results to the screen. I tried to keep these file and console operations separate from the store operations, so it is easier to see where the side effects happen.

3. Which OCaml functions could reasonably be considered pure?

Functions such as set, get, delete, and list are pure. They use the values passed into them and return a result without changing some outside variable or performing file I/O. The transaction function is also pure as long as a pure operation is passed into it. It only decides whether to return the modified or the original store.

4. Where does mutable state exist in the C++ implementation?

The mutable state in the C++ program is the `data_map` inside the Store class. The map changes when commands like SET, DELETE, and LOAD are used. The Transaction class also has a `committed_flag` that changes when `commit()` is called. Unlike the OCaml version, the C++ program changes the same store as it runs.

5. Where does mutable state exist in the OCaml implementation?

There is not really any mutable state in my OCaml code because I do not use ref or mutable fields. My OCaml version is built around an immutable map, so functions like set and delete return a new store instead of changing the old one. The command loop then continues using the new store. The things that do change are outside the store itself, such as the file on disk and the output printed to the screen.

6. How does RAII help control or isolate side effects in C++?

RAII helps manage side effects by tying them to an object's lifetime. For example, `ifstream` and `ofstream` objects open files when they are created and close them automatically when they go out of scope. This means I do not have to remember to manually close the file. The same idea is used in my Transaction class. The constructor saves a backup of the store, and the destructor restores it if `commit()` was never called.

7. How does immutability help control side effects in OCaml?

Immutability helps control side effects because functions cannot change the existing store. Functions like set and delete return a new store instead of modifying the old one. The original

store is still available, and the program has to choose to continue with the new store. This keeps changes to the store from happening unexpectedly.

8. How does rollback work differently in the two implementations?

Rollback works differently in the two languages. In C++, the store is changed directly, so the transaction has to save a backup before making changes. If the transaction is not committed, the destructor copies the backup back into the store. In OCaml, the original store was never changed, so rollback simply means continuing with the original value instead of using the modified one.

9. Why can the OCaml implementation retain an earlier state without explicitly copying the entire map?

OCaml maps are persistent data structures. When a value is added or changed, OCaml does not need to copy the entire map. The new map can reuse parts of the old map that did not change. Because those parts are immutable, it is safe for both versions of the store to share them.

10. Which programming model made transaction rollback easier to express? Explain the mechanism rather than simply stating a preference.

I think OCaml made transaction rollback easier to express. In C++, I had to save a backup of the store, keep track of whether the transaction was committed, and restore the backup if it was not. In OCaml, the original store is still there, so I only use the new store if the operation succeeds. If it fails, the program just keeps using the original store. That made the OCaml version simpler for transaction rollback.